

Data Dictionary

Complete reference for every persisted record in the **VIP Connect External Campaigns** platform.

1. DynamoDB Table: `VipAdminPlans`

Single-table design. Two record kinds share the same partition.

Attribute	Type	Purpose
<code>pk</code>	String	Partition key. Always <code>PLAN#{planId}</code> .
<code>sk</code>	String	Sort key. <code>META</code> for plan, <code>RUN#{epoch_ms}-{uuid8}</code> for run.

Sort key shape `RUN#{epoch_ms}-{uuid8}` gives chronological lexicographic ordering — `ScanIndexForward=False` returns most recent run first.

1.1 Plan META record (`pk=PLAN#{planId}`, `sk=META`)

Field	Type	Required	Description	PHI
<code>planId</code>	String	Yes	UUID4	No
<code>name</code>	String	Yes	Operator-visible name	No
<code>description</code>	String	No	Optional notes	No
<code>trigger</code>	Map	Yes	<code>{type, ...}</code> — see below	No
<code>loop</code>	Map	No	<code>{startTime?, endTime}</code> (COT HH:MM)	No
<code>workingHours</code>	Map	No	<code>{days:[MON,...], startTime, endTime}</code>	No
<code>buckets</code>	List	Yes	Array of <code>BucketDef</code>	No
<code>isTemplate</code>	Boolean	No	Hides plan from active runs queue if true	No
<code>isDefault</code>	Boolean	No	Operator-facing default selection flag	No
<code>pendingWarmup</code>	Map	No	<code>{campaigns:[{campaignId, connectCampaignId, segmentName, segmentArn, warmupStarted}]}</code> — cleared on consumption	No

Field	Type	Required	Description	PHI
runLock	String	No	runId of an active run; blocks concurrent triggers	No
createdAt	String	Yes	ISO 8601 UTC	No
updatedAt	String	Yes	ISO 8601 UTC	No

trigger shapes

Type	Fields
manual	none
time	time:"HH:MM" COT, repeat?:bool
on_plan_complete	planId:string, repeat:bool, afterBucket?:int, afterCampaign?:string

BucketDef

Field	Type	Description
id	String	UUID4
name	String	Display name
run_mode	String	time_based (uses run_duration_minutes) or status_based (advances when all campaigns terminal)
prestart_next	Boolean	Whether to pre-warm next bucket (default true)
parallel	Boolean	If true, starts in parallel with previous bucket
cleanup	Boolean	Whether the bucket cleans up created segments on exit
duration_minutes	Integer	Soft / display value
run_duration_minutes	Integer	Hard time cap; triggers _expire_bucket
reconcileRetryLimit	Integer	Max empty-segment retries (default 5; legacy plans may carry 1 — see TD-005)
onReconcileExhausted	String	continue or fail
campaignConfig	Map	Default Connect config for all campaigns in this bucket
campaigns	List	Array of CampaignDef

CampaignDef

Field	Type	Description
id	String	UUID4
name	String	Display name

Field	Type	Description
states	List[String]	Geographic codes: NY, LI, NJ, CT, MD, TX, NCA, SCA
groups	List[String]	Segment group labels, e.g. "Cancellation / 1st attempt"
run_type	String	full (until daily 19:00 cutoff) or custom (use run_duration_minutes)
run_duration_minutes	Integer	Used when run_type=custom
dependsOn	List[String]	Campaign IDs this must wait for. Empty = stage-1. Cross-bucket allowed (AND semantics).
deliveryType	String	campaign (Connect MANAGED) or journey (Connect JOURNEY)
campaignConfig	Map	Overrides bucket-level config when present
pinnedSegmentArn	String	If set, skip Redis lookup; use this segment ARN directly

campaignConfig:

```
{
  queueId: string,
  contactFlowId?: string,           // optional override; otherwise resolved by name
  sourcePhoneNumber: string,
  dialerType: "PREDICTIVE" | "PROGRESSIVE",
  bandwidthAllocation: float,       // 0.0 .. 1.0
  dialingCapacity: number,
  amdEnabled: boolean,
  amdAwaitPrompt: boolean,
  ringTimeout?: number              // seconds
}
```

1.2 Run record (pk=PLAN#{planId} , sk=RUN#{runId})

Field	Type	Description
runId	String	{epoch_ms}-{uuid8}
planId	String	Parent plan id
status	String	running, completed, cancelled, failed
startedAt	String	ISO 8601 UTC
completedAt	String	ISO 8601 UTC
triggeredBy	String	scheduled, manual, chained
currentBucketIndex	Integer	Index of the bucket currently advancing
planSnapshot	Map	Frozen copy of the Plan at run start; executor reads this, not the live plan

Field	Type	Description
bucketStates	List	Array of BucketState
error	String	Free-text reason if status=failed
scheduleName	String	Legacy run-level rule name (migrated to per-bucket on read)
_version	Integer	Optimistic locking counter, increments on every save

BucketState

Field	Type	Description
bucketId	String	Matches BucketDef.id
name	String	Cached display name
status	String	queued, warming, running, completed, cancelled, expired, error
startedAt	String	Set at activation time (NOT pre-warm time)
completedAt	String	ISO 8601
scheduleName	String	EventBridge rule vip-plan-{runId}-{idx}
campaignStates	List	Array of CampaignState

CampaignState

Field	Type	Description
campaignId	String	Matches CampaignDef.id
name	String	Cached display name
status	String	queued, warming, creating, running, completed, cancelled, expired, error
connectCampaignId	String	Amazon Connect campaign UUID
segmentName	String	Customer Profiles segment definition name (timestamp-encoded)
segmentArn	String	ARN of the segment definition
leadCount	Integer	Estimated lead count when known
startedAt	String	ISO 8601
completedAt	String	ISO 8601
exitReason	String	See enum below
errorDetail	String	Human-readable error if exitReason=error
reconcileRetries	Integer	Count of empty-segment retries
warmupStarted	Boolean	True while pre-warmed; removed after activation
creatingAt	String	

Field	Type	Description
		Set when status enters <code>creating</code> ; used to time out stale claims after 300s

`exitReason` **enum**

Value	Meaning
<code>completed</code>	Connect reported COMPLETED
<code>stopped</code>	StopCampaign called (operator / 19:00 cutoff)
<code>expired</code>	Campaign <code>run_duration_minutes</code> elapsed
<code>bucket_expired</code>	Bucket time cap reached
<code>error</code>	Unrecoverable error; see <code>errorDetail</code>
<code>skipped_empty</code>	Reconcile exhausted; segment had no members
<code>reconcile_failed</code>	Segment did not materialise within retries
<code>creation_failed</code>	CreateCampaign or StartCampaign threw
<code>cancelled</code>	Abort by operator
<code>parent_cancelled</code>	Parent campaign cancelled and dependent treated cancelled
<code>aborted</code>	Run aborted before campaign could start
<code>cutoff_too_close</code>	Less than 6 minutes to 19:00 COT; refused to start

2. PHI Data Map

Field	Where stored	Where logged	Notes
Patient phone number	ElastiCache Redis (<code>wait_list:{team}:list</code>) and within Customer Profiles	Never in CloudWatch	<code>StructuredLogger._scrub</code> replaces <code>phone</code> , <code>phone_number</code> with <code>*_hash</code> (SHA-256 / first 12 chars)
Patient first/last name	Customer Profiles only	Never	<code>first_name</code> , <code>last_name</code> , <code>fullname</code> are scrubbed
<code>lead_id</code>	Internally referenced	Logged as <code>lead_id_hash</code> + 8-char <code>lead_id_prefix</code>	Hash + prefix only
Segment definition name	DynamoDB (<code>segmentName</code>)	Yes (operator name + timestamp, no PHI)	Names are constructed from state code + group + timestamp
Connect campaign ID	DynamoDB (<code>connectCampaignId</code>)	Yes (UUID, no PHI)	OK to log

PHI is **never** in DynamoDB. Names and phone numbers stay inside Connect + Customer Profiles only.

3. Lambda × Table × Operations Matrix

Lambda	<code>WritePlanInPlan</code>	<code>WriteRunInRun</code>	Redis	Customer Profiles	Connect
api-plans	Read+Write (plans, runs)	Write (audit)	Read	Create + Get + Delete + List segments	Create + Start + Stop + Get + Describe + List campaigns; List/ Create contact flows
api-campaigns	—	Write	—	Get + List segments	Full CRUD on campaigns
api-metrics	Read	Read + Write	—	—	—
api-profiles	—	Write	—	CRUD on profiles + segments	—
api-segments	Read	Write	Read	CRUD on segments	—

4. Access Patterns

Pattern	Operation	Cost notes
Get plan by id	<code>GetItem pk=PLAN#{id} sk=META</code>	1 RCU
List plans	<code>Scan FilterExpression sk=META</code>	One scan per call; plan table is < 200 items so acceptable
Get run by id	<code>GetItem pk=PLAN#{id} sk=RUN#{runId}</code>	1 RCU
List recent runs of a plan	<code>Query pk=PLAN#{id} sk begins_with(RUN#) ScanIndexForward=False Limit=20</code>	Cheap
Latest run of plan	Same as above with <code>Limit=1</code>	Cheap
Find plans triggered by another plan	<code>Scan then filter on trigger.type=on _plan_complete AND trigger.planId=...</code>	Scan; plan table small
Optimistic write on run	<code>PutItem ConditionExpression="attribute_not_exists(_version) OR _version = :v"</code>	Throws <code>ConcurrentWriteError</code> on conflict
Set/clear pendingWarmup	<code>UpdateItem SET pendingWarmup or REMOVE pendingWarmup</code>	Single update
Lock plan against concurrent runs	<code>UpdateItem SET runLock=:r ConditionExpression="attribute_not_exists(runLock)"</code>	Throws <code>ValueError</code> if already locked

5. Redis Data

Key pattern	Type	Purpose	PHI
<code>wait_list:{team}:list</code>	Hash	Map of <code>lead_id</code> → JSON payload containing <code>phone_number</code> , <code>first_name</code> , <code>last_name</code> , segment group labels, source state location	YES (phones + names)

- Populated by an **external feeder pipeline** (out of scope for this repo).
- `RedisLeadSource.fetch_for_filters(state_codes, groups)` performs server-side filtering through batched `HSCAN`.
- Lead lists are mid-day refreshable; reconcile retries handle the race where the segment is queried before the feeder catches up.

6. VipAdminAudit Table

Owned by other services but produced by api-plans through

`vip_shared.infrastructure.persistence.audit`:

Field	Type	Notes
<code>pk</code>	String	<code>AUDIT#{entityType}#{entityId}</code>
<code>sk</code>	String	<code>{timestamp}#{uuid}</code>
<code>actorSub</code>	String	Cognito sub
<code>actorEmail</code>	String	Cognito email
<code>action</code>	String	e.g. <code>start</code> , <code>abort</code> , <code>create</code>
<code>before</code>	Map	Optional pre-image
<code>after</code>	Map	Optional post-image
<code>ipAddress</code>	String	From API Gateway
<code>userAgent</code>	String	From API Gateway
<code>timestamp</code>	String	ISO 8601

PHI is never written to audit records.